

Unit-7: I/O Management & Disk Scheduling

Introduction to I/O Management

1. What is I/O Management?

I/O Management is the process of controlling:

- Input devices
- Output devices
- Data transfer between CPU, memory, and devices

The Operating System acts as a manager between:

- Hardware devices
- User programs

2. Why I/O Management is Needed

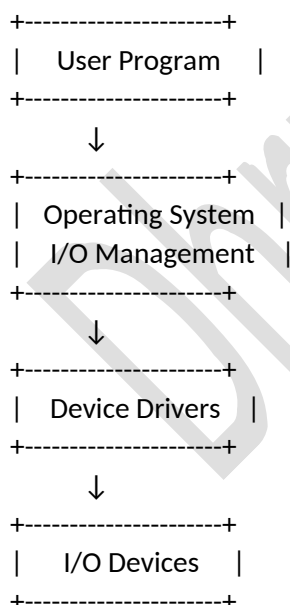
Without I/O management:

- CPU cannot communicate properly with devices
- Data transfer becomes inefficient
- System performance decreases

Main Objectives of I/O Management

Objective	Purpose
Device Control	Manage hardware devices
Efficiency	Improve data transfer speed
Resource Sharing	Multiple programs use devices
Error Handling	Detect and manage device errors
Device Independence	Same interface for different devices

3. Basic I/O System Architecture



4. Working Principle of I/O Management

Step-by-Step Working

Step	Operation
1	Program requests I/O operation

2	OS receives request
3	Device driver activated
4	Hardware device performs operation
5	Data transferred
6	Result returned to program

Example

Printing a Document

Application



Operating System



Printer Driver



Printer

I/O Devices

5. Introduction to I/O Devices

I/O devices are hardware components used for:

- Input
- Output
- Storage
- Communication

They allow interaction between:

- Computer system
- External environment

6. Definition of I/O Device

Term	Definition
I/O Device	Hardware device used for input, output, or data transfer

7. Types of I/O Devices

Main Categories

Category	Purpose
Input Devices	Send data to computer
Output Devices	Receive data from computer
Storage Devices	Store data permanently
Communication Devices	Exchange data between systems

8. Input Devices

Input devices send data to the computer.

Examples of Input Devices

Device	Function
Keyboard	Text input
Mouse	Pointer control
Scanner	Image/document input
Microphone	Audio input

Webcam	Video input
---------------	-------------

Working of Input Device

User Action



Input Device



OS & Driver



CPU Processing

9. Output Devices

Output devices display processed results.

Examples of Output Devices

Device	Function
Monitor	Visual output
Printer	Hard copy output
Speaker	Audio output
Projector	Large display

Working of Output Device

CPU Output



Operating System



Device Driver



Output Device

10. Storage Devices

Storage devices store data permanently.

Examples

Device	Type
HDD	Magnetic storage
SSD	Flash storage
Pen Drive	Portable storage
CD/DVD	Optical storage

Storage Hierarchy

Registers



Cache



RAM



SSD/HDD

11. Communication Devices

Communication devices exchange data between systems.

Examples

Device	Function
Network Card	Network communication
Modem	Internet connection
Router	Data routing

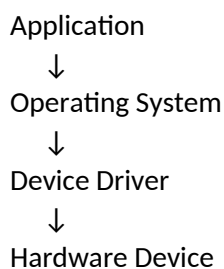
12. Characteristics of I/O Devices

Characteristic	Explanation
Speed	Data transfer rate
Data Format	Binary/text/audio/video
Error Rate	Accuracy of device
Capacity	Storage capability

13. Device Drivers

Definition: A device driver is software that helps the operating system communicate with hardware devices.

Device Driver Architecture



Examples of Drivers

Driver	Hardware
Printer Driver	Printer
Audio Driver	Speaker
Graphics Driver	GPU
Network Driver	Network card

14. Device Controller

Definition: A device controller is hardware responsible for controlling I/O devices.

Functions of Device Controller

Function	Purpose
Receive commands	From CPU/OS
Transfer data	Between device and memory
Error reporting	Notify problems

15. Types of I/O Operations

Operation Type	Description
Read Operation	Input data from device

Write Operation	Output data to device
------------------------	-----------------------

Example

Device	Read	Write
Keyboard	Yes	No
Monitor	No	Yes
Hard Disk	Yes	Yes

16. Synchronous vs Asynchronous I/O

Feature	Synchronous I/O	Asynchronous I/O
CPU Waits?	Yes	No
Speed	Slower	Faster
Complexity	Simple	Complex
Efficiency	Lower	Higher

17. Interrupts in I/O Devices

What is Interrupt?

An interrupt is a signal sent by hardware to CPU for immediate attention.

Interrupt Working

I/O Device



Interrupt Signal



CPU Stops Current Task



Service Device Request

Examples of Interrupts

Device	Interrupt Example
Keyboard	Key pressed
Printer	Printing completed
Disk	Data transfer completed

18. DMA (Direct Memory Access)

Definition: DMA allows devices to transfer data directly to memory without continuous CPU involvement.

DMA Architecture

I/O Device



DMA Controller



Main Memory

Advantages of DMA

Advantages
Faster data transfer
Reduced CPU workload

Better system performance

Disadvantages of DMA

Disadvantages

Complex

hardware

Higher cost

19. Common I/O Problems

Problem	Explanation
Speed Mismatch	CPU faster than devices
Device Failure	Hardware malfunction
Buffer Overflow	Excess incoming data
Interrupt Overload	Too many interrupts

20. Advantages of I/O Management

Advantages
Efficient device handling
Better performance
Resource sharing
Device independence
Error management

21. Disadvantages of I/O Management

Disadvantages
Complex implementation
Device dependency issues
Interrupt overhead

22. Real-World Examples

Area	I/O Device Usage
ATM Machine	Keyboard, display, printer
Mobile Phone	Touchscreen, speaker, camera
Web Server	Disk and network devices
Gaming System	Controller, GPU, audio

23. Key Terms

Term	Meaning
I/O Device	Hardware for input/output
Device Driver	Software for hardware communication
Interrupt	Hardware attention signal
DMA	Direct Memory Access
Device Controller	Hardware managing device
Buffer	Temporary data storage

24. Important Exam Points

Most Important Questions

1. Explain I/O management.
2. Explain types of I/O devices.
3. Explain device drivers.
4. Explain interrupts in I/O systems.
5. Explain DMA with diagram.
6. Differentiate synchronous and asynchronous I/O.
7. Explain role of operating system in I/O management.

25. Short Conclusion

I/O management is responsible for controlling communication between the operating system and hardware devices. Input, output, storage, and communication devices work together with device drivers, interrupts, and DMA mechanisms to ensure efficient data transfer and system performance. Proper I/O management improves resource utilization, speed, and reliability in computer systems.

Organization of I/O Functions

According to the GTU Operating Systems syllabus, "Organization of I/O Functions" is an important topic in Unit-7: I/O Management & Disk Scheduling.

1. Introduction

I/O organization defines how the operating system manages communication between:

- CPU
- Memory
- I/O devices

I/O functions are organized to:

- Improve performance
- Reduce CPU workload
- Handle multiple devices efficiently

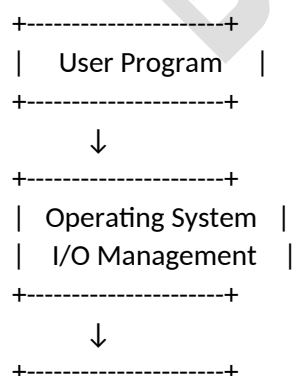
The operating system coordinates:

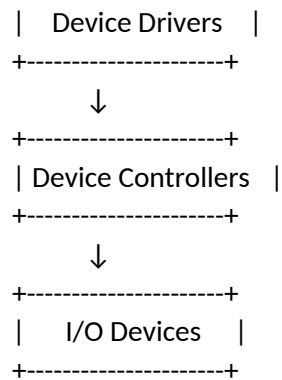
- Device communication
- Data transfer
- Interrupt handling
- Buffer management

2. Definition

Term	Definition
I/O Function Organization	Structured method used by OS to manage input/output operations and devices

3. Basic I/O Organization Architecture





4. Working Principle of I/O Organization

Step-by-Step Working

Step	Operation
1	User program requests I/O
2	OS receives request
3	Device driver activated
4	Device controller controls hardware
5	Data transferred
6	Completion signal returned

Example

Reading Data from Disk

Application



Operating System



Disk Driver



Disk Controller



Hard Disk

5. Components of I/O Organization

Main components:

1. I/O Devices
2. Device Controllers
3. Device Drivers
4. Interrupt Handlers
5. Buffers
6. DMA Controller

6. I/O Devices

Definition

Hardware devices used for:

- Input
- Output
- Storage

- Communication

Examples

Type	Devices
Input	Keyboard, Mouse
Output	Monitor, Printer
Storage	HDD, SSD
Communication	Network card

7. Device Controller

Definition: A device controller is hardware responsible for controlling I/O devices.

It acts as:

- Interface between device and CPU

Functions of Device Controller

Function	Description
Control Device	Operates hardware
Transfer Data	Device ↔ Memory
Error Detection	Detects hardware errors
Status Reporting	Reports device state

Device Controller Diagram

CPU



Device Controller



I/O Device

Example

Device	Controller
Hard Disk	Disk Controller
Keyboard	Keyboard Controller
USB Device	USB Controller

8. Device Drivers

Definition: A device driver is software that enables communication between:

- Operating system
- Hardware device

Device Driver Working

Operating System



Device Driver



Hardware Device

Functions of Device Drivers

Function	Purpose
Device Communication	Control hardware

Error Handling	Detect problems
Data Transfer	Send/receive data
Device Initialization	Start device

Examples of Drivers

Driver	Hardware
Printer Driver	Printer
Audio Driver	Speaker
Graphics Driver	GPU

9. Interrupt Handlers

Definition: Interrupt handler is a routine executed when:

- Hardware sends interrupt signal

Interrupt Working

I/O Device



Interrupt Signal



CPU Stops Current Task



Interrupt Handler Executes

Functions of Interrupt Handler

Function	Purpose
Respond to Device	Handle request
Resume CPU	Continue execution
Improve Efficiency	Avoid CPU waiting

Example

Keyboard key press generates:

- Keyboard interrupt

10. I/O Buffering

Definition: Buffering is temporary storage of data during transfer.

Buffers reduce:

- Speed mismatch between CPU and devices

Buffering Diagram

I/O Device



Buffer



Main Memory

Need for Buffering

Need	Explanation
Speed Difference	CPU faster than devices
Smooth Transfer	Continuous data flow

Reduce Interrupts	Improves performance
--------------------------	----------------------

Types of Buffering

Type	Description
Single Buffering	One buffer used
Double Buffering	Two buffers alternate
Circular Buffering	Multiple rotating buffers

11. DMA (Direct Memory Access)

Definition: DMA allows direct data transfer between:

- I/O device
- Main memory

without continuous CPU involvement.

DMA Architecture

I/O Device



DMA Controller



Main Memory

DMA Working

Step	Operation
1	CPU initializes DMA
2	DMA controls transfer
3	Data moved directly
4	DMA interrupts CPU after completion

Advantages of DMA

Advantages
Faster transfer
Lower CPU overhead
Efficient large transfers

Disadvantages of DMA

Disadvantages
Complex hardware
Expensive implementation

12. Polling Technique

Definition: In polling, CPU repeatedly checks device status.

Polling Diagram

CPU



Check Device Status



Device Ready?

Characteristics of Polling

Feature	Description
CPU Waits	Yes
Simplicity	Simple implementation
Efficiency	Lower

Advantages

Advantages
Easy to implement
Suitable for simple systems

Disadvantages

Disadvantages
CPU time wasted
Slow response

13. Interrupt-Driven I/O

Definition: CPU executes other tasks until device sends interrupt.

Interrupt-Driven I/O Diagram

CPU Working
↓
Interrupt Arrives
↓
Interrupt Handler Runs

Advantages

Advantages
Better CPU utilization
Faster response
Efficient multitasking

Disadvantages

Disadvantages
Complex management
Interrupt overhead

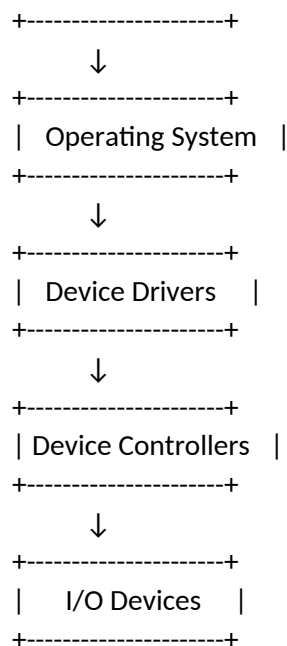
14. Comparison of I/O Techniques

Polling vs Interrupt vs DMA

Feature	Polling	Interrupt	DMA
CPU Involvement	High	Medium	Low
Efficiency	Low	Medium	High
Speed	Slow	Faster	Very Fast
Complexity	Simple	Moderate	Complex

15. Organization of Complete I/O System

+-----+
| User Process |



16. Operating System Design Issues in I/O

Design Issue	Explanation
Device Independence	Same interface for all devices
Error Handling	Manage hardware failures
Efficiency	Faster transfer
Resource Sharing	Multiple users/processes
Security	Access protection

17. Real-World Examples

System	I/O Organization Example
Printer System	Driver + interrupt handling
Hard Disk	DMA transfer
Keyboard	Interrupt-driven input
Network Card	Buffered communication

18. Advantages of Organized I/O Functions

Advantages
Better performance
Efficient CPU usage
Faster data transfer
Device independence
Reduced waiting time

19. Disadvantages

Disadvantages
Complex implementation
Hardware dependency
Interrupt overhead

20. Key Terms

Term	Meaning
Device Driver	Software controlling hardware
Device Controller	Hardware controlling device
Interrupt	Hardware signal to CPU
DMA	Direct Memory Access
Buffer	Temporary storage
Polling	Continuous status checking

21. Important Exam Points

Most Important Questions

1. Explain organization of I/O functions.
2. Explain device controller and device driver.
3. Explain interrupt-driven I/O.
4. Explain DMA with diagram.
5. Differentiate polling and interrupt.
6. Explain buffering techniques.
7. Draw and explain I/O organization architecture.

22. Short Conclusion

The organization of I/O functions in an operating system ensures efficient communication between CPU, memory, and hardware devices. Components such as device drivers, controllers, interrupts, buffers, and DMA work together to improve performance, reduce CPU workload, and manage multiple I/O operations efficiently.

Operating System Design Issues in I/O Management

1. Introduction

The operating system must efficiently manage:

- I/O devices
- Data transfer
- Resource sharing
- Device communication

While designing I/O systems, several challenges arise related to:

- Performance
- Security
- Device handling
- CPU utilization

These challenges are called:

- Operating System Design Issues

2. Definition

Term	Definition
OS Design Issues in I/O	Problems and considerations faced while designing efficient input/output management in an operating system

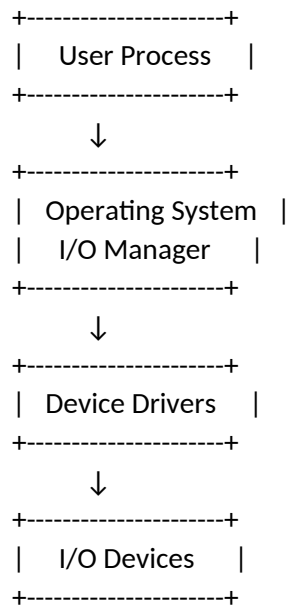
3. Goals of I/O System Design

The operating system aims to:

Goal	Purpose
------	---------

Efficiency	Faster I/O operations
Reliability	Accurate data transfer
Device Independence	Uniform device handling
Security	Protected access
Resource Sharing	Multiple process support

4. Basic Architecture of I/O Design



5. Main Operating System Design Issues

Major I/O design issues are:

1. Device Independence
2. Error Handling
3. Efficiency & Performance
4. Buffering
5. Caching
6. Scheduling
7. Protection & Security
8. Resource Sharing
9. Interrupt Handling
10. Deadlock Handling

6. Device Independence

6.1 Introduction

Different devices have different hardware structures.

The operating system should provide:

- Common interface for all devices

Applications should work without knowing:

- Internal hardware details

Example

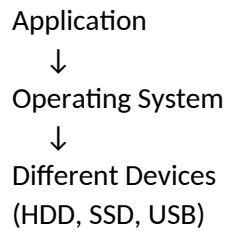
A text editor should work with:

- HDD

- SSD
- USB drive

without changing program code.

Device Independence Diagram



Advantages

Advantages
Easy programming
Hardware flexibility
Portability

Disadvantages

Disadvantages
Extra abstraction overhead

7. Error Handling

7.1 Introduction

I/O devices may generate errors such as:

- Device failure
- Read/write failure
- Network disconnection

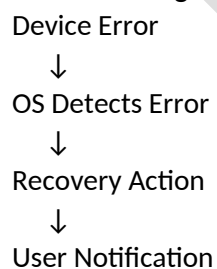
OS must:

- Detect errors
- Recover from failures
- Notify users

Types of Errors

Error Type	Example
Hardware Error	Disk failure
Communication Error	Network timeout
Data Error	Corrupted file

Error Handling Process



Advantages

Advantages
Improves reliability
Prevents data loss

Disadvantages

Disadvantages
Additional processing overhead

8. Efficiency and Performance

8.1 Introduction

I/O devices are usually slower than CPU.

OS must optimize:

- Data transfer
- CPU utilization
- Device usage

Performance Techniques

Technique	Purpose
Buffering	Reduce speed mismatch
Caching	Faster access
DMA	Reduce CPU load
Scheduling	Efficient request handling

Example

Using:

- Disk cache
- DMA transfer

improves disk performance.

Advantages

Advantages
Faster execution
Better CPU utilization

Disadvantages

Disadvantages
Complex implementation

9. Buffering

9.1 Introduction

Buffering stores data temporarily during transfer.

It helps manage:

- Speed difference between CPU and devices

Buffering Diagram

I/O Device



Buffer



Main Memory

Types of Buffering

Type	Description
Single Buffer	One temporary buffer
Double Buffer	Two alternate buffers
Circular Buffer	Multiple rotating buffers

Advantages

Advantages
Smooth transfer
Reduced waiting time

Disadvantages

Disadvantages
Extra memory required

10. Caching

10.1 Introduction

Cache stores frequently used data for faster access.

Cache Working

CPU
↓
Cache
↓
Main Memory
↓
Disk

Example

Recently opened files stored in:

- Disk cache

Advantages

Advantages
Faster access
Reduced disk operations

Disadvantages

Disadvantages
Cache consistency problems

11. Scheduling of I/O Requests

11.1 Introduction

Many processes may request devices simultaneously.

OS schedules requests to:

- Improve performance
- Reduce waiting time

Example

Disk scheduling algorithms:

- FCFS
- SSTF
- SCAN
- C-SCAN

Scheduling Diagram

Multiple Requests



I/O Scheduler



Optimized Execution

Advantages

Advantages

Better throughput

Reduced seek time

Disadvantages

Disadvantages

Complex algorithms

12. Protection and Security

12.1 Introduction

OS must protect devices and data from:

- Unauthorized access
- Malicious programs
- Illegal operations

Security Mechanisms

Mechanism	Purpose
Permissions	Access control
Authentication	Verify users
Encryption	Data protection

Example

Only administrator allowed to:

- Format disk
- Install drivers

Advantages

Advantages

Data safety

Secure operation

Disadvantages

Disadvantages

Additional overhead

13. Resource Sharing

13.1 Introduction

Multiple users/processes may need same device.

OS manages:

- Fair sharing
- Synchronization
- Access control

Example

Printer shared by:

- Multiple users

Resource Sharing Diagram

Process 1

Process 2

Process 3



Operating System



Shared Printer

Advantages

Advantages

Efficient resource usage

Cost reduction

Disadvantages

Disadvantages

Synchronization problems

14. Interrupt Handling

14.1 Introduction

I/O devices send interrupts to CPU when:

- Operation completed
- Error occurs

OS must:

- Handle interrupts efficiently

Interrupt Handling Process

Device



Interrupt Signal



CPU



Interrupt Handler

Advantages

Advantages
Better CPU utilization
Faster response

Disadvantages

Disadvantages
Interrupt overhead

15. Deadlock Handling

15.1 Introduction

Deadlock occurs when:

- Processes wait indefinitely for devices/resources

Example

Process A waiting for Printer

Process B waiting for Scanner

Deadlock Prevention Techniques

Technique	Purpose
Resource Allocation	Controlled assignment
Resource Ordering	Fixed order
Timeout	Prevent infinite waiting

Advantages

Advantages
Stable system
Avoids resource blocking

Disadvantages

Disadvantages
Reduced flexibility

16. Complete View of I/O Design Issues

+-----+	
I/O Design Issues in OS	
+-----+	
Device Independence	
Error Handling	
Buffering	
Caching	
Scheduling	
Security	
Resource Sharing	
Interrupt Handling	
Deadlock Handling	
+-----+	

17. Comparison Table

Buffering vs Caching

Feature	Buffering	Caching
Purpose	Smooth transfer	Faster access
Data Type	Temporary transfer data	Frequently used data
Speed Improvement	Moderate	High

Polling vs Interrupt

Feature	Polling	Interrupt
CPU Wait	Yes	No
Efficiency	Lower	Higher
Complexity	Simple	Moderate

18. Real-World Examples

System	Design Issue Example
Printer System	Resource sharing
Hard Disk	Scheduling
USB Devices	Device independence
Cloud Servers	Security and buffering
Database Systems	Caching

19. Advantages of Good I/O Design

Advantages
High performance
Better reliability
Efficient resource usage
Improved security
Reduced waiting time

20. Disadvantages

Disadvantages
Complex implementation
Hardware dependency
Increased OS size

21. Key Terms

Term	Meaning
Buffer	Temporary storage
Cache	Fast memory for frequent data
Interrupt	Hardware attention signal
Scheduling	Managing request order
Deadlock	Infinite waiting condition
Device Driver	Software controlling hardware

23. Short Conclusion

Operating system design issues in I/O management involve handling performance, security, buffering, caching, scheduling, interrupts, and resource sharing efficiently. Proper design improves system reliability, CPU utilization, data transfer speed, and overall operating system performance.

I/O Buffering

According to the GTU Operating Systems syllabus, "I/O Buffering" is an important topic in Unit-7: I/O Management & Disk Scheduling.

1. Introduction

In computer systems:

- CPU works very fast
- I/O devices are comparatively slow

Because of this speed difference:

- CPU may remain idle while waiting for I/O operations

To solve this problem:

- Buffering is used

I/O buffering improves:

- Data transfer efficiency
- CPU utilization
- System performance

2. Definition

Term	Definition
Buffer	Temporary memory area used to store data during transfer
I/O Buffering	Technique of temporarily storing data while transferring between devices and memory

3. Need for I/O Buffering

I/O buffering is required because:

Problem	Solution by Buffering
CPU faster than devices	Temporary storage
Speed mismatch	Smooth data transfer
Frequent interrupts	Reduced CPU interruption
Slow I/O operations	Continuous processing

4. Basic Working Principle of I/O Buffering

Data Transfer Flow

I/O Device
↓
Buffer Memory
↓
Main Memory / CPU

Step-by-Step Working

Step Operation

- 1 Device sends data
- 2 Data stored in buffer
- 3 CPU processes buffered data
- 4 New data continues arriving

Example

Keyboard Input

Keyboard



Keyboard Buffer

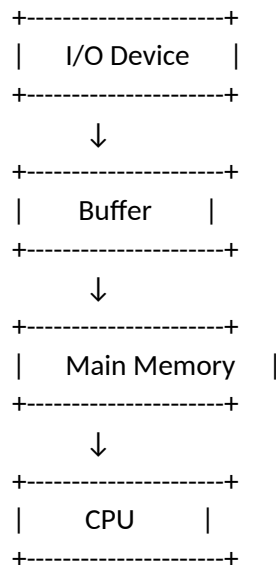


CPU Reads Characters

Characters typed by user are first stored in:

- Keyboard buffer

5. I/O Buffering Architecture



6. Functions of I/O Buffering

Function	Purpose
Temporary Storage	Hold data temporarily
Speed Matching	Match CPU/device speeds
Reduce Interrupts	Fewer CPU interruptions
Improve Throughput	Faster data handling
Continuous Transfer	Smooth communication

7. Types of I/O Buffering

Main types:

1. Single Buffering
2. Double Buffering
3. Circular Buffering

8. Single Buffering

8.1 Definition

In single buffering:

- Only one buffer is used between device and memory.

8.2 Working

I/O Device



Single Buffer

↓
CPU

8.3 Working Principle

Step	Operation
1	Device fills buffer
2	CPU processes buffer
3	Device waits until buffer becomes free

8.4 Advantages

Advantages
Simple implementation
Low memory requirement

8.5 Disadvantages

Disadvantages
CPU/device waiting possible
Lower performance

8.6 Real-World Example

- Basic keyboard buffering

9. Double Buffering

9.1 Definition

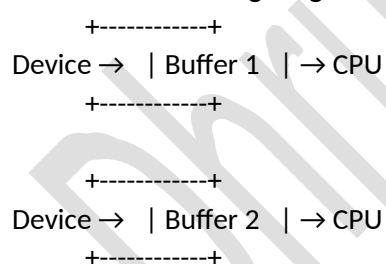
In double buffering:

- Two buffers are used alternately.

While:

- One buffer processed by CPU
- Other buffer filled by device

9.2 Double Buffering Diagram



9.3 Working Principle

Step	Operation
1	Device fills Buffer-1
2	CPU processes Buffer-1
3	Simultaneously device fills Buffer-2
4	Alternating operation continues

9.4 Advantages

Advantages
Better performance

Continuous processing

Reduced waiting time

9.5 Disadvantages

Disadvantages

More memory required

More complex management

9.6 Real-World Example

- Video streaming
- Audio playback

10. Circular Buffering

10.1 Definition

Circular buffering uses:

- Multiple buffers arranged in circular form

Data continuously rotates through buffers.

10.2 Circular Buffer Diagram

+-----+
| Buffer1 |

+-----+
↓

+-----+
| Buffer2 |

+-----+
↓

+-----+
| Buffer3 |

+-----+
↓

(Return to Buffer1)

10.3 Working Principle

Step	Operation
1	Device fills buffers sequentially
2	CPU processes buffers continuously
3	Buffer cycle repeats

10.4 Real-World Example

- Network streaming
- Multimedia systems

11. Buffering vs Caching

Feature	Buffering	Caching
Purpose	Temporary transfer storage	Faster repeated access
Data Type	Transfer data	Frequently used data
Usage	I/O operations	Performance optimization
Memory Duration	Short-term	Longer-term

12. Buffering vs Spooling

Feature	Buffering	Spooling
Storage Size	Small temporary memory	Large disk area
Purpose	Smooth transfer	Queue management
Example	Keyboard buffer	Printer queue

13. Advantages of I/O Buffering

Advantages
Improved CPU utilization
Faster I/O operations
Reduced waiting time
Smooth data transfer
Reduced interrupt frequency

14. Disadvantages of I/O Buffering

Disadvantages
Extra memory requirement
Complex management
Possible buffer overflow

15. Buffer Overflow

Definition

Buffer overflow occurs when:

- Incoming data exceeds buffer capacity

Example

Buffer Size = 100 bytes

Incoming Data = 150 bytes

Extra data may:

- Corrupt memory
- Crash system

Prevention Methods

Method	Purpose
Proper buffer size	Avoid overflow
Error checking	Detect problems
Flow control	Manage transfer rate

16. Operating System Role in Buffering

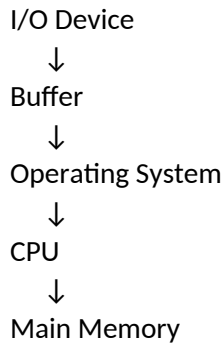
The operating system:

- Allocates buffers
- Controls transfer
- Synchronizes devices
- Handles interrupts
- Prevents overflow

17. Real-World Examples of Buffering

System	Buffer Usage
Keyboard	Character buffering
Printer	Print buffering
Video Streaming	Frame buffering
Audio Player	Sound buffering
Network Router	Packet buffering

18. Complete Buffering Workflow



19. Comparison Table of Buffering Types

Feature	Single	Double	Circular
Buffers Used	1	2	Multiple
Performance	Low	Medium	High
Complexity	Simple	Moderate	High
Memory Usage	Low	Medium	High

20. Key Terms

Term	Meaning
Buffer	Temporary storage area
Throughput	Amount of processed data
Buffer Overflow	Exceeding buffer capacity
Circular Queue	Rotating buffer arrangement
Synchronization	Coordinated processing

22. Short Conclusion

I/O buffering is an important technique used to reduce speed mismatch between CPU and I/O devices. By temporarily storing data during transfer, buffering improves system efficiency, CPU utilization, and overall performance. Techniques such as single, double, and circular buffering are widely used in modern operating systems and multimedia systems.

Disk Scheduling

1. Introduction

A hard disk stores data in:

- Tracks
- Sectors

When multiple disk requests arrive:

- Operating system decides the order of servicing requests

This process is called:

- Disk Scheduling

Disk scheduling improves:

- Disk performance
- CPU utilization
- Response time

2. Definition

Term	Definition
Disk Scheduling	Technique used by operating system to decide the order of servicing disk I/O requests

3. Need for Disk Scheduling

Many processes may request disk access simultaneously.

Without scheduling:

- Disk head movement increases
- Seek time increases
- Performance decreases

Main Objectives

Objective	Purpose
Reduce Seek Time	Faster disk access
Improve Throughput	More requests processed
Reduce Waiting Time	Faster response
Improve Efficiency	Better performance

4. Disk Structure Basics

Hard Disk Components

Hard Disk
Tracks
Sectors
Read/Write Head
Disk Arm

Important Terms

Term	Meaning
Track	Circular path on disk
Sector	Smallest storage unit
Disk Head	Reads/writes data
Seek Time	Time to move disk head
Rotational Delay	Waiting for sector rotation

5. Working Principle of Disk Scheduling

Basic Working

Disk Requests Queue



Disk Scheduling Algorithm

↓
Optimized Head Movement
↓
Data Access

Example Queue

Request Queue: 98, 183, 37, 122, 14, 124, 65, 67
Initial Head Position: 53

6. Types of Disk Scheduling Algorithms

Main algorithms:

1. FCFS (First Come First Serve)
2. SSTF (Shortest Seek Time First)
3. SCAN
4. C-SCAN

FCFS (First Come First Serve) Disk Scheduling

FCFS is the simplest disk scheduling algorithm used in operating systems for handling disk I/O requests. It services requests in the exact order they arrive in the queue.
This algorithm is included in Unit-7: I/O Management & Disk Scheduling.

1. Definition

Term	Definition
FCFS (First Come First Serve)	Disk scheduling algorithm where requests are serviced in arrival order

2. Basic Idea

The disk head processes requests:

- One by one
- In queue order
- Without optimization

It follows:

- FIFO (First In First Out) principle

3. Working Principle

Steps

Step	Operation
1	Requests enter queue
2	First request serviced first
3	Next requests processed sequentially
4	Continues until queue becomes empty

4. FCFS Diagram

Example Queue

Initial Head Position = 53

Request Queue:
98, 183, 37, 122, 14

Head Movement

53 → 98 → 183 → 37 → 122 → 14

5. Example Calculation

Request Sequence

Order	Request
Start	53
1	98
2	183
3	37
4	122
5	14

Total Head Movement

Movement	Distance
53 → 98	45
98 → 183	85
183 → 37	146
37 → 122	85
122 → 14	108

Total Seek Time

45 + 85 + 146 + 85 + 108 = 469 tracks

6. Characteristics of FCFS

Characteristic	Description
Scheduling Type	Non-preemptive
Queue Policy	FIFO
Complexity	Very simple
Optimization	No optimization

7. Advantages of FCFS

Advantages
Easy to understand
Simple implementation
Fair scheduling
No starvation problem

8. Disadvantages of FCFS

Disadvantages
Large seek time
Poor disk performance
High waiting time
Unnecessary head movement

9. Real-World Analogy

FCFS works like:

- People standing in a ticket counter queue

The person arriving first:

- Gets service first

10. Performance Analysis

Factor	FCFS Performance
Seek Time	High
Throughput	Low
Waiting Time	High
Fairness	Good

11. Comparison with SSTF

Feature	FCFS	SSTF
Selection Method	Arrival order	Nearest request
Performance	Lower	Better
Complexity	Simple	More complex
Starvation	No	Possible

12. Applications of FCFS

Area	Usage
Simple operating systems	Basic scheduling
Small systems	Low workload environments
Teaching algorithms	Basic OS concepts

13. Key Terms

Term	Meaning
Seek Time	Disk head movement time
FIFO	First In First Out
Request Queue	Waiting disk requests
Head Movement	Movement of disk arm

15. Short Conclusion

FCFS is the simplest disk scheduling algorithm where requests are serviced in the order they arrive. Although it is fair and easy to implement, it suffers from high seek time and lower performance because it does not optimize disk head movement.

SCAN Disk Scheduling Algorithm

SCAN is an important disk scheduling algorithm used in operating systems to reduce disk head movement and improve performance. It is also known as the:

- Elevator Algorithm

This topic is included in Unit-7: I/O Management & Disk Scheduling.

1. Definition

Term	Definition
SCAN Disk Scheduling	Algorithm where disk head moves in one direction servicing requests and then reverses direction

2. Basic Idea

The disk head:

- Moves in one direction

- Services all requests in its path
- Reaches end of disk
- Reverses direction

This movement is similar to:

- Elevator movement in a building

3. Working Principle

Step-by-Step Process

Step	Operation
1	Disk head moves in one direction
2	Requests in path are serviced
3	Head reaches disk end
4	Direction reversed
5	Remaining requests serviced

4. SCAN Movement Diagram

Example

Initial Head Position = 53

Direction = Left

Request Queue: 98, 183, 37, 122, 14, 124, 65, 67

Head Movement

14 ← 37 ← 53 → 65 → 67 → 98 → 122 → 124 → 183

5. SCAN Algorithm Example

Request Queue

Requests
98
183
37
122
14
124
65
67

Initial Head: 53

Direction: Left

Service Order

53 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

6. Characteristics of SCAN

Characteristic	Description
Scheduling Type	Direction-based
Head Movement	Back and forth
Performance	Better than FCFS
Starvation	Rare

7. Advantages of SCAN

Advantages
Reduced seek time
Better throughput
Efficient disk usage
Less waiting time
Better performance than FCFS

8. Disadvantages of SCAN

Disadvantages
More complex than FCFS
Edge requests may wait longer
Direction reversal overhead

9. Why SCAN is Called Elevator Algorithm

The disk head movement resembles:

- Elevator movement

Just like elevator:

- Moves floor-by-floor in one direction
- Then reverses direction

Disk head:

- Services requests similarly

Elevator Analogy

Elevator: Ground → Top → Ground

SCAN: Start → End Track → Reverse

10. Performance Analysis

Factor	SCAN Performance
Seek Time	Lower
Throughput	Better
Waiting Time	Moderate
CPU Efficiency	High

11. SCAN vs FCFS

Feature	FCFS	SCAN
Head Movement	Random	Directional
Performance	Lower	Better
Seek Time	High	Lower
Complexity	Simple	Moderate

12. SCAN vs SSTF

Feature	SCAN	SSTF
Starvation	Rare	Possible
Fairness	Better	Lower
Complexity	Moderate	Moderate

13. Real-World Applications

Area	Usage
Operating Systems	Disk management
Database Servers	Faster storage access
Cloud Systems	Large-scale storage
File Servers	Efficient disk scheduling

14. Complete SCAN Workflow

Disk Requests



SCAN Algorithm



Head Moves in One Direction



Requests Serviced



Direction Reversed

18. Short Conclusion

SCAN disk scheduling improves disk performance by moving the disk head systematically in one direction and servicing requests along the path before reversing direction. It reduces seek time, improves throughput, and provides better efficiency compared to FCFS scheduling.

C-SCAN (Circular SCAN) Disk Scheduling

C-SCAN is an advanced disk scheduling algorithm designed to provide:

- Uniform waiting time
- Better fairness
- Efficient disk access

This algorithm is included in Unit-7: I/O Management & Disk Scheduling.

1. Definition

Term	Definition
C-SCAN (Circular SCAN)	Disk scheduling algorithm where disk head moves in only one direction and returns to beginning after reaching end

2. Basic Idea

In C-SCAN:

- Disk head moves in one direction only
- Services requests along the path
- Reaches end of disk
- Jumps back to start
- Continues servicing again

Unlike SCAN:

- Head does not service requests while returning

3. Working Principle

Step-by-Step Process

Step	Operation
1	Head moves in one direction

2	Requests are serviced
3	Head reaches end track
4	Head returns directly to beginning
5	Process repeats

4. C-SCAN Movement Diagram

Example

Initial Head Position = 53

Direction = Right

Request Queue:

98, 183, 37, 122, 14, 124, 65, 67

Head Movement

53 → 65 → 67 → 98 → 122 → 124 → 183

↓

Return

↓

0 → 14 → 37

5. Example of C-SCAN Scheduling

Request Queue

Requests
98
183
37
122
14
124
65
67

Initial Head Position: 53

Direction: Right

Service Order

53 → 65 → 67 → 98 → 122 → 124 → 183

→ Return to Start →

14 → 37

6. Characteristics of C-SCAN

Characteristic	Description
Head Movement	One direction only
Return Method	Direct jump to beginning
Fairness	Better than SCAN
Waiting Time	More uniform

7. Why C-SCAN is Called Circular SCAN

The head movement forms:

- Circular pattern

Start → End → Jump Back → Start

This circular servicing gives:

- Equal chance to all requests

8. Advantages of C-SCAN

Advantages
Uniform waiting time
Better fairness
Reduced starvation
Efficient for heavy loads
Predictable response time

9. Disadvantages of C-SCAN

Disadvantages
Extra head movement
More complex implementation
Return jump adds overhead

10. Real-World Analogy

C-SCAN works like:

- Circular bus route

The bus:

- Moves in one direction
- Completes full route
- Returns to starting point

11. Performance Analysis

Factor	C-SCAN Performance
Seek Time	Low
Throughput	High
Fairness	Very Good
Waiting Time	Uniform

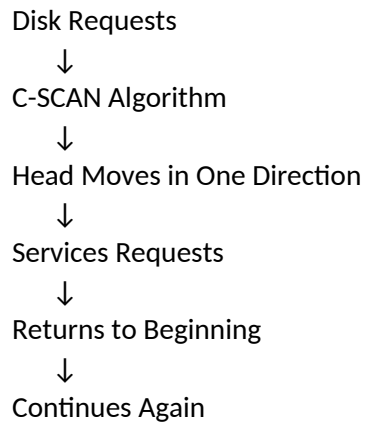
12. C-SCAN vs SCAN

Feature	SCAN	C-SCAN
Direction	Both directions	One direction
Return Path	Services while returning	Direct return
Fairness	Moderate	Better
Waiting Time	Uneven	Uniform

13. C-SCAN vs FCFS

Feature	FCFS	C-SCAN
Performance	Lower	Better
Seek Time	High	Lower
Complexity	Simple	Higher
Fairness	Moderate	Better

14. Complete Workflow of C-SCAN



15. Applications of C-SCAN

Area	Usage
Database Servers	Uniform response time
Multuser Systems	Fair scheduling
Cloud Storage	Heavy disk workload
Large File Servers	Continuous request handling

19. Short Conclusion

C-SCAN disk scheduling improves fairness and efficiency by servicing requests in only one direction and returning directly to the beginning after reaching the end of the disk. It provides uniform waiting time, better throughput, and improved performance for heavily loaded systems.

SSTF (Shortest Seek Time First) Disk Scheduling

SSTF is an important disk scheduling algorithm used in operating systems to reduce:

- Seek time
- Disk head movement
- Waiting time

This topic is included in Unit-7: I/O Management & Disk Scheduling.

1. Definition

Term	Definition
SSTF (Shortest Seek Time First)	Disk scheduling algorithm that services the request nearest to current disk head position first

2. Basic Idea

In SSTF:

- The disk head selects the nearest request
- Minimum head movement is performed
- Seek time is reduced

The algorithm always chooses:

- Closest track from current position

3. Working Principle

Step-by-Step Process

Step	Operation
1	Disk requests enter queue

2	Current head position checked
3	Nearest request selected
4	Head moves to nearest track
5	Process repeats

4. SSTF Working Diagram

Example Queue

Initial Head Position = 53

Request Queue:

98, 183, 37, 122, 14, 124, 65, 67

Head Movement

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

5. Example of SSTF Scheduling

Request Queue

Requests
98
183
37
122
14
124
65
67

Initial Head Position: 53

Step-by-Step Selection

Current Position	Nearest Request
53	65
65	67
67	37
37	14
14	98
98	122
122	124
124	183

Final Sequence

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

6. Characteristics of SSTF

Characteristic	Description
Scheduling Type	Distance-based
Request Selection	Nearest request
Performance	Better than FCFS
Starvation	Possible

7. Advantages of SSTF

Advantages
Reduced seek time
Better throughput
Faster response time
Improved disk performance
Efficient head movement

8. Disadvantages of SSTF

Disadvantages
Starvation possible
Complex implementation
Long waiting for distant requests

9. Starvation Problem in SSTF

Definition: Starvation occurs when:

- Some requests wait indefinitely because nearby requests continuously get priority.

Example

If head remains near: 50–70 tracks

then distant request:

180

may wait very long.

10. Real-World Analogy

SSTF works like:

- Elevator serving nearest floor request first

Nearest request gets:

- Immediate priority

11. Performance Analysis

Factor	SSTF Performance
Seek Time	Low
Throughput	High
Waiting Time	Lower
Fairness	Lower than FCFS

12. SSTF vs FCFS

Feature	FCFS	SSTF
Request Selection	Arrival order	Nearest request
Seek Time	High	Lower
Performance	Lower	Better
Starvation	No	Possible

13. SSTF vs SCAN

Feature	SSTF	SCAN
Head Movement	Nearest request	Direction-based

Starvation	Possible	Rare
Fairness	Lower	Better
Complexity	Moderate	Moderate

14. Complete Workflow of SSTF

Disk Requests



Current Head Position



Nearest Request Selected



Disk Head Moves



Process Repeats

15. Applications of SSTF

Area	Usage
Operating Systems	Disk scheduling
Database Systems	Faster data access
File Servers	Reduced response time
Storage Systems	Performance optimization

16. Comparison of Disk Scheduling Algorithms

Algorithm	Performance	Starvation	Complexity
FCFS	Low	No	Simple
SSTF	Good	Yes	Moderate
SCAN	Better	Rare	Moderate
C-SCAN	Very Good	No	Higher

19. Short Conclusion

SSTF disk scheduling improves disk performance by selecting the request nearest to the current disk head position. It reduces seek time and increases throughput compared to FCFS scheduling. However, distant requests may suffer starvation due to continuous preference for nearby requests.

RAID (Redundant Array of Independent Disks)

According to the GTU Operating Systems syllabus, RAID is an important topic in Unit-7: I/O Management & Disk Scheduling.

1. Introduction to RAID

RAID is a technology used to:

- Improve disk performance
- Increase storage reliability
- Provide data protection

Instead of using:

- One disk

RAID combines:

- Multiple disks into one logical unit

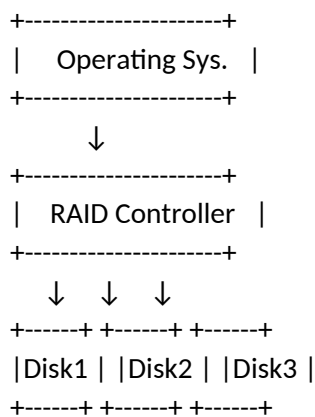
2. Definition

Term	Definition
RAID	Technique that combines multiple physical disks for performance, reliability, or fault tolerance

3. Objectives of RAID

Objective	Purpose
Fault Tolerance	Protect data from disk failure
High Performance	Faster data access
Large Storage	Combine multiple disks
Reliability	Reduce data loss

4. RAID Architecture



5. Working Principle of RAID

Basic Working

Step	Operation
1	Data divided into blocks
2	Blocks distributed across disks
3	Redundant/parity data stored
4	Data recovered if disk fails

6. Important RAID Concepts

Concept	Meaning
Striping	Dividing data across disks
Mirroring	Copying same data to multiple disks
Parity	Extra information for recovery
Fault Tolerance	Ability to survive disk failure

7. RAID Levels

Main RAID levels:

1. RAID 0
2. RAID 1
3. RAID 5
4. RAID 6
5. RAID 10

8. RAID 0 (Striping)

Definition: Data divided across multiple disks without redundancy.

RAID 0 Diagram

Data Blocks:

A1 → Disk1

A2 → Disk2

A3 → Disk3

Usage

- Gaming systems
- Video editing

9. RAID 1 (Mirroring)

Definition: Same data copied to multiple disks.

RAID 1 Diagram

Disk1 → Data A

Disk2 → Data A

Usage

- Banking systems
- Critical servers

10. RAID 5

Definition: Uses:

- Striping + parity

Parity distributed across disks.

RAID 5 Diagram

Disk1 → Data

Disk2 → Data

Disk3 → Parity

Usage

- Enterprise servers
- Database systems

11. RAID 6

Definition: Similar to RAID 5 but stores:

- Double parity

Can survive:

- Two disk failures

Advantages

Advantages

Very high reliability

Better fault tolerance

Disadvantages

Disadvantages

More storage overhead

Higher complexity

12. RAID 10 (1+0)

Definition: Combination of:

- RAID 1 + RAID 0

Uses:

- Mirroring + striping

RAID 10 Diagram

Mirrored Pairs



Striped Together

Advantages

Advantages

Very high speed

High reliability

Disadvantages

Disadvantages

Expensive

Requires many disks

13. Comparison of RAID Levels

RAID Level	Speed	Fault Tolerance	Storage Efficiency
RAID 0	Very High	No	High
RAID 1	Medium	High	Low
RAID 5	High	Medium	Good
RAID 6	Medium	Very High	Moderate
RAID 10	Very High	High	Moderate

14. Advantages of RAID

Advantages

Faster performance

Better reliability

Data protection

Increased storage capacity

Fault tolerance

15. Disadvantages of RAID

Disadvantages

Expensive setup

Complex management

Extra hardware required

16. Real-World Applications of RAID

Area

Usage

Cloud Servers	Data protection
Banking Systems	Reliability
Data Centers	High storage
Database Servers	Fast access

17. Key Terms of RAID

Term	Meaning
Striping	Data division across disks
Mirroring	Data duplication
Parity	Recovery information
Fault Tolerance	Disk failure handling

18. Important Exam Points for RAID

Frequently Asked Questions

1. Define RAID.
2. Explain RAID 0 and RAID 1.
3. Explain RAID 5.
4. Differentiate RAID levels.
5. Explain striping and mirroring.
6. Write advantages and disadvantages of RAID.

19. Short Conclusion of RAID

RAID combines multiple disks to improve storage performance, reliability, and fault tolerance. Different RAID levels provide different balances between speed, storage efficiency, and data protection.

Disk Cache

20. Introduction to Disk Cache

Disk cache is a temporary high-speed memory used to store:

- Frequently accessed disk data

It improves:

- Disk access speed
- System performance

21. Definition

Term	Definition
Disk Cache	Temporary memory storing recently or frequently used disk data

22. Need for Disk Cache

Hard disks are slower than:

- RAM
- CPU

Disk cache reduces:

- Frequent disk access

23. Disk Cache Architecture

CPU

↓

Main Memory

↓
Disk Cache
↓
Hard Disk

24. Working Principle of Disk Cache

Step-by-Step Process

Step	Operation
1	CPU requests data
2	OS checks disk cache
3	If found → fast access
4	If not found → read from disk
5	Data stored in cache

Example

Frequently opened file

↓
Stored in Cache
↓
Faster access next time

25. Types of Disk Cache

Type	Description
Read Cache	Stores frequently read data
Write Cache	Temporarily stores write data

26. Cache Hit and Cache Miss

Term	Meaning
Cache Hit	Data found in cache
Cache Miss	Data not found in cache

Example

Cache Hit → Fast Access

Cache Miss → Disk Access Needed

27. Advantages of Disk Cache

Advantages
Faster data access
Reduced disk operations
Improved performance
Better CPU efficiency

28. Disadvantages of Disk Cache

Disadvantages
Extra memory required
Cache consistency issues
Data loss risk during power failure

29. Disk Cache vs Buffer

Feature	Disk Cache	Buffer
Purpose	Faster repeated access	Temporary transfer storage
Data Type	Frequently used data	Transfer data
Speed Benefit	High	Moderate

30. Real-World Applications of Disk Cache

Area	Usage
Operating Systems	Faster file access
Web Browsers	Cached web files
Databases	Query optimization
Servers	Performance improvement

31. Key Terms of Disk Cache

Term	Meaning
Cache Hit	Data found in cache
Cache Miss	Data absent in cache
Read Cache	Stores read data
Write Cache	Stores write operations

33. Short Conclusion of Disk Cache

Disk cache improves operating system performance by storing frequently used disk data in fast temporary memory. It reduces disk access time, improves throughput, and increases overall system efficiency.